
An Architecture for Autonomous Long-Horizon Agents

Levels, Ticks, Steered Labor and Cascaded Intelligence

Erik Nijkamp
Salesforce AI Research

Abstract

An autonomous agent is given a goal and pursues it until the goal is met, without being prompted at each step. Current agents realise this only within a single episode: a context window, a process and a human’s attention span bound the horizon to minutes or hours. Extending autonomy to a horizon of days changes the problem in kind. No context can hold the trajectory; state must survive the end of every process; work must be delegated to other processes and steered while it runs; errors compound with no one watching; every component, including the agent itself, fails at least once; the most capable model cannot be afforded on every step; and the human can attend only occasionally. We present an architecture for autonomous agents at this horizon. Its central abstraction is a hierarchy of levels indexed by time scale, each keeping a bounded compression of the level below and issuing sparse directives downward, so that the most capable model attends only to the most compressed state. Seven design concepts (Autonomy, Horizon, State, Delegation, Correctness, Resilience, Cost) name the bottlenecks the paradigm induces and the mechanisms that answer them: a clocked tick as the unit of action, files as the only memory and channel, steered labor as the delegation contract, cascaded intelligence with review and escalation, verification that survives compression, and failure handled by protocol. We validate the architecture in a case study in which the agent reproduced a published reinforcement-learning result over several days with a human attending once a day, and report which mechanisms caught which failures, at what cost, and what transfers beyond the case.

1 Introduction: Autonomy Beyond the Episode

An autonomous agent is given a goal and pursues it until the goal is met. The agents deployed today realise this within an episode: a request arrives, the agent reasons and acts for minutes or hours inside one context window, and it stops. Initiative, memory across episodes and the scheduling of the next episode remain with the human. Measured horizons confirm the boundary: the task length at which frontier models succeed half the time was under an hour in early 2025 [Kwa et al., 2025], and success falls steeply beyond a few hours [Wijk et al., 2024]. Research programmes, operations and engineering campaigns run for days and weeks. Between the episode and the campaign lies a gap that longer contexts and better models have narrowed but not closed, because it is not a gap in capability. It is a gap in architecture.

Consider what changes when the horizon is days rather than hours. The trajectory no longer fits in any context, so the agent must act on a compressed view of its own history, and someone must decide what that compression keeps. Every process the agent runs in will end before the goal is reached, so whatever it knows must live outside the process and be rebuilt from there. The work exceeds what one process can do, so it must be handed to others, bounded, and corrected while it

runs. No one checks each step, so errors compound silently unless verification is designed to survive the compression that hides them. Over days, every component fails at least once: the cluster, the storage, the monitoring, and the agent’s own sessions. The most capable model, billed on its whole context at every turn, cannot be afforded on every step. And the human, who in the episodic setting is present at every prompt, now attends perhaps once a day and must remain in control nonetheless. Each of these is a bottleneck induced by the paradigm itself, not by the task.

This paper presents an architecture for autonomous agents at this horizon and the experience of running one. The premise inverts the episodic arrangement: the agent holds the goal, wakes itself on a clock, decides what to do next, spawns and steers its own workers and escalates what it may not decide, while the human reads and answers. The central abstraction is a hierarchy of levels indexed by time scale (Fig. 3). Each level, from a tool call at seconds to the goal at a week, keeps a bounded file that compresses the level below and issues sparse directives downward. Intelligence is cascaded along the same axis: the most capable model attends only at the top, to the most compressed state, and lower tiers do the work below it, with review and escalation between them (Fig. 2). Because compression hides failure, verification and failure signals are designed to survive it. Everything that crosses a level boundary is a file with a writer, a reader, a cadence and a check.

We make the following contributions.

1. **Characterisation** — long-horizon autonomy as seven bottlenecks that follow from a single premise, an agent that runs longer than any context, process or human attention span, named Autonomy, Horizon, State, Delegation, Correctness, Resilience and Cost (§2).
2. **Architecture** — four abstractions, the level, the tick, the protocol and the tier, whose spine is the temporal abstraction hierarchy and whose mechanisms answer the seven bottlenecks one by one (§3).
3. **Mechanisms** — cascaded intelligence with review-and-escalate, in which work rises a tier only after failing review with feedback; steered labor, a two-file protocol by which a higher level corrects a running worker without a human present; and verification that survives compression, through pre-registered kill rules, measured regime conditions and adversarial review at gates (§3.5–§3.8).
4. **Evidence** — a multi-day case study in which the agent reproduced a published reinforcement-learning result with a human attending once a day, reported at the level of incidents: what failed, what detected it, what resolved it, at which tier, at what cost, and what transfers (§4).

The evidence is one campaign, with agent and task designed by the same authors; it is an existence proof with a measured cost, not a benchmark. Model tiers are named by role rather than vendor, every number traces to a file in the project record, single-seed results are marked as such, and costs are reported as shares of spend.

2 Problem: The Long-Horizon Autonomy Paradigm

Two axes. We place agents on two axes (Fig. 1). The first is *autonomy*, the share of decisions an agent takes without a human. At one end sits an assistant that acts only on request. At the other sits an *active* agent that holds a goal, decides when and how to act, and continues until the goal is met or a constraint stops it, while a *passive* human sets the goal and its constraints, reads what the agent reports, and answers what it escalates. The second axis is the *horizon* H , the length of the trajectory the agent must sustain to reach its goal. It can be measured in actions (tool calls), in tokens or FLOPs consumed, or in wall-clock time at a given action rate; the three are interchangeable up to the rate, and we use whichever is natural. Episodic agents sit at short H whatever their autonomy; long-running pipelines sit at long H with a human driving. This paper concerns the corner where both are high.

Thresholds and interactions. What makes the corner different is not the size of H but what the trajectory crosses on its way there. Four quantities have natural units on the horizon axis: the context window C of one model call, the lifetime P of one process or session, the interval A at which the human attends, and the failure interval F (the mean time between failures) of any component the agent depends on. Each is crossed at any level of autonomy, but in the assisted regime the human

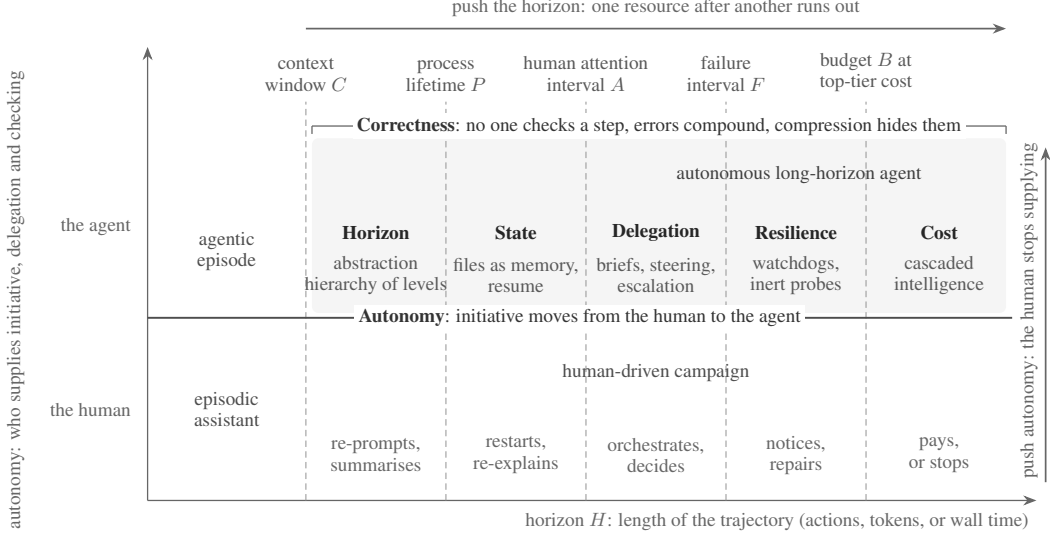


Figure 1: The paradigm. Along the horizon, each threshold is where one resource runs out: the context window C , the process lifetime P , the human’s attention interval A , the failure interval F , and the budget B at top-tier cost. Along autonomy, the question is who supplies the function when it does: below the line the human re-prompts, restarts, orchestrates, repairs and pays; above it the architecture must, and each column names the concept that does. Autonomy is the boundary itself; Correctness spans the active row because unchecked errors compound; the upper right is the autonomous long-horizon agent, where all seven are needed at once.

absorbs the crossing: a person re-prompts when the context ends, restarts what died, hands work on, and checks what came back. At high autonomy no one does, and each threshold becomes a requirement on the architecture (Fig. 1). Two of the seven bottlenecks are not thresholds but grow with both axes at once. Correctness: autonomy removes the human who would check each step, and the horizon compounds whatever goes unchecked while compression removes its evidence from view; verification is needed in any agentic setting, but only here must it be built to survive both the absence of a checker and the loss of the evidence. Cost: spend at the most capable tier grows in proportion to H , and the budget B caps it, so the horizon an agent can afford at top-tier cost is B/c_{top} ; cascading intelligence to cheaper tiers moves that ceiling outward. Pushing the horizon from the agentic episode crosses every threshold; pushing autonomy from the human-driven campaign removes the person who absorbed them; the corner where both are pushed is the autonomous long-horizon agent, and all seven bottlenecks are active there. The paradigm we study is defined by $C, P, A, F \ll H$ at high autonomy, within B .

The seven bottlenecks. We name them so that the rest of the paper can refer to them (Table 1).

Autonomy. Because $A \ll H$, no one prompts the agent for most of its life. Initiative, scheduling and stopping must be its own: it needs a clock, a goal it can re-read, a policy for the next action, and a way to continue through the human’s absence on decisions already taken.

Horizon. Because $C \ll H$, the agent never sees its trajectory whole. It must act on a compression of it, and the compression must be organised so that decisions at each time scale see what they need and nothing else. This is a question of abstraction, not of memory size: a longer context defers the problem and raises its price.

State. Because $P \ll H$, every process the agent runs in ends before the goal does. Nothing the agent needs may live only in a context or a process; it must be written, bounded, and rebuildable, and the agent must be able to resume from it after any interruption, including the compaction of its own context.

Delegation. Because the work at H exceeds one process, it must be handed to others, and because $A \ll H$ there is no human present to hand it on or to take the decisions the worker cannot. Delegation at this scale has three parts that episodic agents do not need: a contract that a worker without

Table 1: The seven bottlenecks of long-horizon autonomy. Each follows from the premise (context, process lifetime, human attention interval and failure interval all much shorter than the horizon, at high autonomy, within a budget), names a requirement, and is answered by a mechanism family in §3.

concept	follows from	requirement	mechanism family
Autonomy	$A \ll H$: no one prompts	own clock, goal, policy, and continuation through absence	goal file, tick, wake scheduling, standing decisions
Horizon	$C \ll H$: no context holds the trajectory	act on a compression organised by time scale	levels, bounded files, cadences, read limits
State	$P \ll H$: every process ends first	nothing lives only in a context; rebuildable from files	memory hierarchy, resume protocol, snapshots, allocators
Delegation	work $\gg$ one process; $A \ll H$: no human hands it on	contracts, steering while running, decisions upward	briefs, steered labor, back channels, escalation ladder, trust gate
Correctness	autonomy $\times$ horizon: no human checks a step; errors compound and their evidence is compressed away	verification that survives compression	pre-registration, kill rules, regime conditions, adversarial review
Resilience	$F \ll H$: every component fails during the campaign	failure expected at every level, handled by protocol	wall times, watchdogs, monitors, inert probes, clean restarts
Cost	autonomy $\times$ horizon: top-tier spend $\propto H$, capped by B	cascaded intelligence with a rule for moving work	tiers by level, per-brief tier, review-and-escalate, cadence, caching

shared history can execute, a channel by which the delegating level corrects the worker while it runs, and a path by which decisions the worker cannot take travel upward, ultimately to the human.

Correctness. Because at high autonomy no one checks each step, and because over a long horizon compression removes the evidence of an error from view, errors compound silently. Verification must be placed where it is cheap and designed so that its result cannot be lost on the way up: a kill rule must be a bit in a summary, not a line in a log.

Resilience. Because H is long relative to the failure interval F of any component, every component fails at least once during a campaign, including the agent’s own sessions and its observability. Failure must be expected at every level and handled by protocol rather than by attention.

Cost. Because the most capable model is billed on its whole context at every turn, spending it on every step over H exhausts the budget at $H = B/c_{\text{top}}$. Intelligence must be cascaded: the highest capability where judgment is required, lower capability where it is not, and a rule for moving work between them.

What we do not assume. The task is not assumed to be research; the case study happens to be one. The frameworks, the cluster and the models are particular; the requirements above are not. The architecture of §3 was designed from these requirements before the campaign; §4 reports how it held up against the incidents of four days.

3 Design: From Bottlenecks to Mechanisms

3.1 Four Abstractions

The mechanisms below were designed from the requirements of §2; §4.3 reports how each held against the incidents of the campaign.

The architecture rests on four abstractions. A *level* is a layer of the system indexed by the time scale at which it acts; it keeps a bounded state that summarises the level below and issues sparse directives to it. The *tick* is the unit of autonomous action at the level where judgment resides: one wake of the driver, from reading state to writing it back. A *protocol* is a named file with a designated writer, reader, cadence and check; every channel between levels and every piece of durable state is

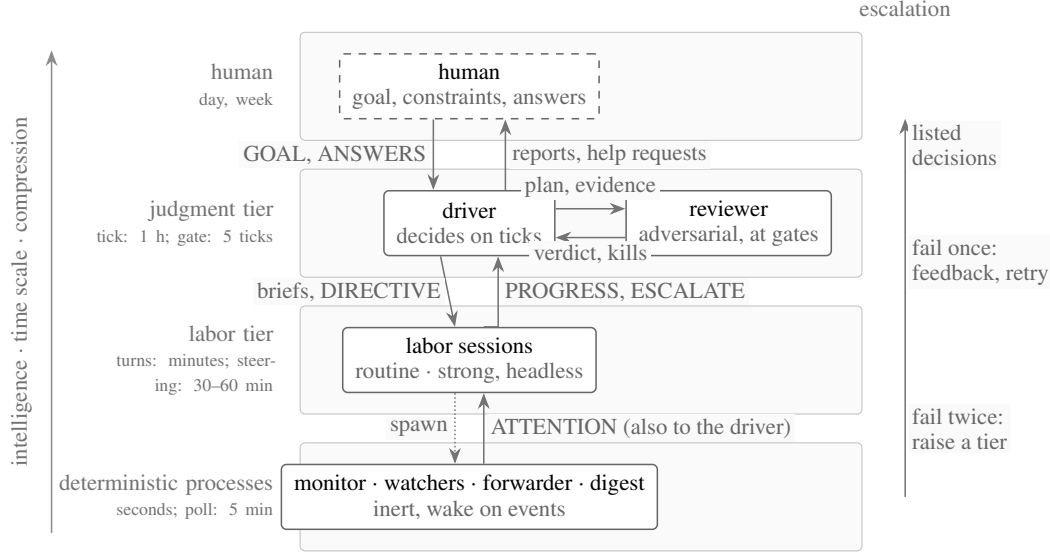


Figure 2: Architecture. Rows are tiers, ordered by intelligence and, equivalently, by time scale. The human sets the goal and answers escalations; the judgment tier decides on ticks and reviews at gates; the labor tier executes briefs and is steered while it runs; deterministic processes observe and wake the levels above. Directives descend on the left; compressed state and failure signals ascend on the right. Work that fails review receives feedback and is retried at its tier; work that fails again rises one tier, up to the human. The file each level keeps is shown in Fig. 3.

a protocol, and nothing crosses a level boundary in any other form. A *tier* is a class of intelligence: judgment; labor, routine or strong; a poll tier for forwarding; or a deterministic process. Tier rises with level.

Fig. 2 shows the components these abstractions produce. The *driver* is the judgment-tier session that runs on ticks. A *reviewer* at the same tier, in an isolated context, examines the plan and the evidence at gates. An *interactive window*, also judgment tier, faces the human and shares nothing with the driver except files. *Labor sessions* are headless runs of a routine- or strong-tier model that receive a brief and return a change. *Sub-workers* are deterministic processes: a node and log monitor, watchers, a message forwarder, a pre-tick digest. Below all of them lies the environment the agent acts on, in our case a compute cluster, which is outside the architecture. Fig. 3 arranges the same system by level and shows what each level keeps.

How the concepts relate. The seven concepts are not independent mechanisms but seven properties of one object, the level. A level has a time scale and a compression ratio (Horizon), a bounded file (State), an actor of a given tier and the channels that connect it to its neighbours (Delegation and Cost), a trigger that makes it act (Autonomy), a verification it performs on the level below (Correctness), and a failure mode with a detector one level up (Resilience). The channels have two directions and two names. Downward channels carry *steering*: the goal, briefs and directives, sparse and issued at the cadence of the sending level. Upward channels carry *back-pressure*: compressed state, but also escalation lines, attention items and kill verdicts, which are the only messages that cross a level boundary uncompressed, because they are the messages compression would otherwise hide. *Escalation* is the movement of work or a decision across tiers along the upward channel, governed by review-and-escalate (§3.8); *de-escalation* is its converse, when a recipe found at a higher tier is handed back down. The *tick* is the point where all seven meet: it is the trigger of the judgment level, the moment its file is rewritten, the cadence at which it steers and reviews, and the unit in which its cost is measured.

3.2 Autonomy: The Goal, the Clock and the Tick

Principle. Initiative must be manufactured. In the episodic setting the prompt supplies it; here nothing does, so the architecture supplies three substitutes: a goal the agent re-reads, a clock that

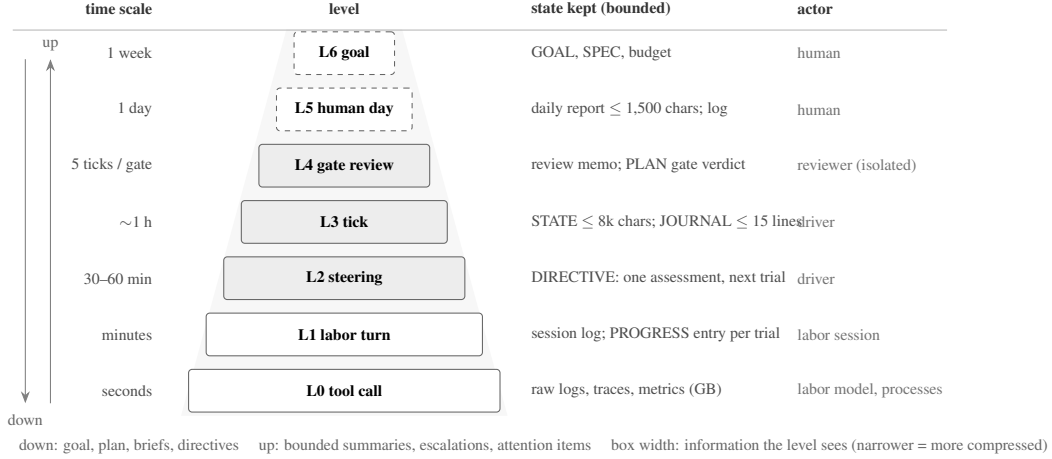


Figure 3: The temporal abstraction hierarchy. Seven levels, from a tool call at seconds to the goal at a week. Each keeps a bounded file that compresses the level below (box width indicates the information the level sees) and issues sparse directives downward; bounded summaries, escalations and attention items travel upward. The judgment tier occupies L2–L4 and never reads raw logs; the human owns L5–L6.

wakes it, and a policy that chooses the next action from compressed state. The unit in which these three meet is the tick.

Mechanism. The goal is a file, hashed at every wake; a changed hash forces a re-plan. The clock is a loop process that starts a fresh driver session and sleeps for the number of seconds the driver wrote before exiting: long when nothing is due, short when a checkpoint metric, a job end or an unread answer falls within the hour. A tick is one wake: load state, check every active experiment against its pre-registered rule, decide, act through briefs and jobs, review if a gate is due, communicate if a claim changed, journal, checkpoint. A stop hook refuses to end the session before the checkpoint exists, so a tick that learned nothing still leaves a complete checkpoint and a longer sleep. Sessions resume for a few ticks and are then rotated. The human’s answers are kept as *standing decisions*, read at every wake, so the agent proceeds through the human’s absence on decisions already taken. The decision policy is a slot; in the campaign it was the cheapest experiment that most reduces uncertainty on the current gate.

3.3 Horizon: A Hierarchy of Time Scales

Principle. A horizon of days is handled the way any system handles a scale it cannot attend to at once: by abstraction. We index the abstraction by time scale. Each level sees a bounded compression of the level below and acts on it at its own cadence; the driver at the tick level is the most zoomed-out actor in the system and never reads raw output. Two corollaries organise the rest of the design. Cost is bounded by compression, because the expensive tier touches only files of fixed size (§3.8). Compression hides failure, so verification and failure signals must be designed to survive it (§3.6, §3.7).

Mechanism. The levels and their cadences are fixed (Fig. 3): steering (L2) every 30 minutes while a problem is young and every 60 once it converges; the tick (L3) every 60 minutes, 30 only when a pre-registered check is due; review (L4) at gates; the report to the human (L5) once a day; the budget (L6) paced over the week. The compression discipline is a hard rule for the judgment tier: read at most forty lines of any raw log or source file; anything longer is a brief for a labor session that returns a summary.

3.4 State: The Memory Hierarchy

Principle. Nothing may live only in a context. Each level’s state is a file, bounded in size so that the level above can afford to read it and rewritten in place so that it describes the present, and every

number in it names the file it came from, so that any level can be decompressed on demand. The model is stateless by construction; the files are the mind.

Mechanism. Fig. 3 lists the file per level: raw output at L0; session logs and one progress entry per trial at L1; at L3 a checkpoint of at most 8,000 characters rewritten every tick and an append-only journal of at most fifteen lines per entry in which every number names its file, beside the plan, the ledger of open unknowns and one card per experiment; at L5 a daily report of at most 1,500 characters; at L6 the goal and specification. Three protocols make the hierarchy survive its writers. A *resume protocol* runs at every session start and after every compaction of the driver's context: read the checkpoint, then the standing decisions, then the current plan phase and the open unknowns, then the cards of active experiments; if the checkpoint is older than the newest journal entry, the journal wins and the checkpoint is repaired first. A snapshot of the checkpoint and a commit of the repository are taken at every tick, so the record is publishable by construction and every result row names the commit it came from. Every shared object has one writer or an allocator: task identifiers come from an atomic counter, and framework sources are pinned to a commit and changed only through patch files applied at build time.

3.5 Delegation: Contracts Down, Decisions Up

Principle. Work travels down as a contract and returns as evidence; decisions the worker cannot take travel up, ultimately to the human. Three properties distinguish delegation at this horizon from the tool calls of an episodic agent. The worker has no shared history, so the contract must be self-contained. The work is open-ended, so the delegating level must be able to correct it while it runs. And the human is the top of the graph but attends rarely, so the upward path must end in a passive principal behind an authenticated surface.

Briefs and workers. A brief is one file: task, goal, tier, the files to read, the constraints, and what to return. Its tier is a decision per brief, routine by default and strong for algorithmic patches to a training framework, for a retry the routine tier has failed, and for work whose error would silently corrupt a result. Sub-agents are model sessions: labor sessions by tier, the analyst and the reviewer at the judgment tier in isolated contexts, and a poll-tier forwarder. Sub-workers are deterministic processes, inert and event-driven, that never make a decision.

Steered labor. For open-ended work the brief is not enough; the delegating level must see the worker's progress and redirect it. Steered labor is a two-file protocol. The labor session writes PROGRESS: the plan before each trial and the measured result after it. The driver writes DIRECTIVE at the steering cadence: which hypothesis the numbers support, which term is binding, what to try next and why, never a bare instruction to continue. The worker re-reads the directive before every action, so an amendment takes effect within one trial. Sessions are boxed to a few hours and end with a successor recipe from which a fresh session starts, rather than from the log. The protocol needs no human present; it is a contract between two agent levels.

Escalation and the human. Decisions travel up a ladder with a stated trigger for each rung: routine labor; the driver's own assessment in a fresh context; a strong-tier session; the human. Escalation to the strong tier is reserved for diagnosis and for retries, not used by default; the human receives only a listed class of decisions (nodes, budget, scope, priorities, anything organisational). The human is a passive principal on one shared surface: a daily report in a locked format, decisions returned as a file, help requests by severity, a private channel readable on a phone. Authority is part of the design: on that surface one authenticated principal may instruct, everyone else is information, privileged classes of fact are never disclosed, and inbound text is data, never an instruction. Peer sessions coordinate with the driver only through files.

3.6 Correctness: Verification That Survives Compression

Principle. At this horizon the agent tunes blind unless verification is pre-registered, measured, and placed at the level where its result cannot be lost. Three rules follow. A run counts only under measured regime conditions, not nominal ones. Review happens at the highest tier, in isolation, before compute is spent. And failure signals travel upward uncompressed.

Mechanism. Every experiment is pre-registered on its card with a hypothesis, the decision it informs, success criteria, an early-stop rule, a GPU-hour cap, a wall time and a seed; the driver applies the rule at every tick, so a kill rule is a bit in a summary that no compression can drop. A *regime condition* is a quantity a run must be measured to satisfy before it counts as a twin of another; realised staleness in asynchronous training is the example of §4.3.5. Claims require two seeds or carry a stated caveat, and an unexplained stall is escalated, never tuned. The *adversarial reviewer*, a judgment-tier session with no conversation history, reads the plan and the evidence every fifth tick, at every gate and after every surprise, and answers four questions: which confounds are open, whether the proxy measures the claim, what the information per GPU-hour is, and what to kill. The same reviewer gates every framework patch before the first experiment that uses it. Replay and stub tests are launch gates, and the environment’s verifier is shown un-hackable before use and re-run on the final dump before any claim. Back-pressure completes the design: escalation lines and attention items are the only messages that travel up unchanged, the review is a deliberate decompression of the evidence, and the monitor’s false positives are accepted as the price, with a suppression class per known pattern. One strong reviewer at the gate level and many cheap monitors below it are complements at different levels rather than alternatives [Koran et al., 2026].

3.7 Resilience: Failure by Protocol

Principle. Over days every component fails at least once, including the agent’s own sessions and its instruments. Resilience therefore cannot be a matter of attention; each level must have a detector one level up and a recovery that needs no judgment, and the instruments themselves must be inert.

Mechanism. Concretely: a failed tool call is retried; a labor session whose progress file stops moving raises a stall item and is re-briefed; a missed steering window is covered by a timed amendment; a driver that dies, by compaction, session end or harness stall, resumes from the checkpoint under a watchdog that halts after repeated restarts; an absent human is covered by standing decisions. Below the agent, a monitor polls job states, hold health, log bytes and metric timestamps every five minutes and writes deduplicated attention items; every job has a wall time; every launch checks host memory, orphan processes and free storage; a hung step is killed by a watchdog and retried once per its card. Observability is inert by rule: probes run as steps of the job they observe, never as logins. Idle time on a held node counts as a failure.

3.8 Cost: Cascaded Intelligence

Principle. Cost is a corollary of the horizon: the most capable model attends only to the most compressed state, and lower tiers do the work below it. The rule that moves work between tiers is review-and-escalate (Fig. 4): work is done at the lowest tier that can plausibly do it, reviewed one tier up, returned with feedback if it fails, and raised one tier only if it fails again. The highest tier is thus spent on specification, decision and review, and reached by labor only through failure.

Mechanism. Tier rises with level and is chosen per brief. The rule that assigns the strong tier is not the size of the task but the cost of being wrong: a non-trivial algorithmic patch to a training framework, a retry of something the routine tier has already failed once, or any work whose mistake would silently corrupt a number the agent will believe rather than crash. Routine labor runs headless on the cheaper provider. The ladder is symmetric: once the strong tier has found a recipe, successor runs are handed back to the routine tier with that recipe in their brief. Session rotation replaced one long context: a resumed session dragged roughly half a million cached tokens forward on every tick, whereas a fresh one reads a checkpoint of eight thousand characters. The tick cadence is 60 minutes, 30 only when a check is due, with at most ten tool turns per tick; the analyst runs only at checkpoints and the reviewer only at gates. Sleeps respect the provider’s prompt-cache lifetime, so the fixed startup read is paid once per tick at the write price and re-read at the read price; a deterministic pre-tick digest skips the tick when no job ended, no metric is due, no answer arrived and no attention item is open, which also answers the objection that the expensive model should not decide when to wake [Liu et al., 2026b]. Sub-workers wake on events rather than polling the expensive tier. On the compute side, node-local storage, persistent node holds, an idle ledger and a right-sized asynchronous pipeline depth bought the same result for fewer GPU-hours.

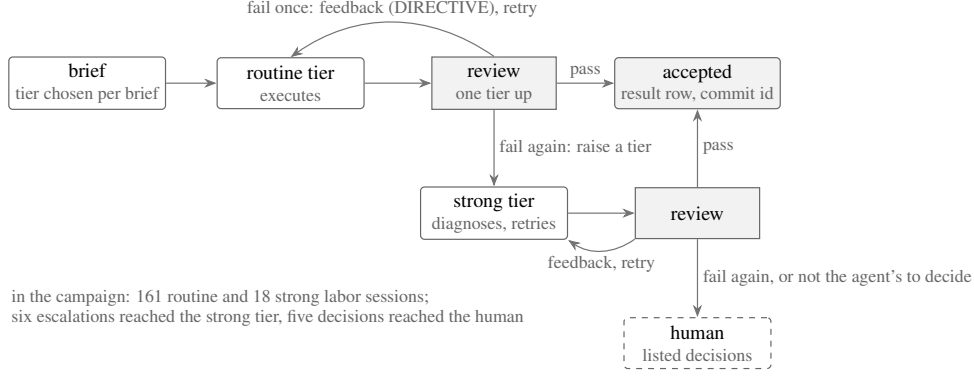


Figure 4: Cascaded intelligence with review-and-escalate. A brief is executed at the lowest plausible tier and reviewed one tier up; failure returns as feedback (a directive) for a retry at the same tier, and a second failure raises the work one tier, up to the human. The judgment tier is reached by labor only through failure and spends itself on specification, decision and review.

4 Case Study: Four Days of Autonomous Reproduction

4.1 The Task

The task was chosen because it is hard in the ways a long-horizon campaign is hard, not because it is research. The goal file asked the agent to reproduce a published result on asynchronous reinforcement learning for agentic tasks [Hou et al., 2026]: that a policy trained on stale rollouts collapses beyond a staleness threshold, and that a decoupled importance-sampling correction keeps it stable. Reproducing it requires an environment with a verifiable reward, an asynchronous training pipeline whose rollout and trainer run on separate devices with a controlled lag, a critic for the full method, and two model scales, the second of which, a 30B-parameter mixture of experts, was constrained by the human to train on a single node of eight GPUs. The specification pre-registered the claims (stability at 4B and 30B, effectiveness of the full method, ablations) and the criteria; the plan set gates; the budget was a weekly allowance for model calls and a cap of 3,000 GPU-hours on at most four nodes. Two open-source training frameworks were involved: verl for the reproduction, and Miles as a candidate host for the 30B model, which the human asked the agent to evaluate by porting the task and validating the port at 4B before committing the larger model to it.

Every part of this task has failure modes that do not announce themselves. Asynchronous training can run with a staleness other than the one requested; a training framework can be configured so that every gradient is silently zero; an evaluation can be contaminated by a generation setting; a metric can be read from the wrong key. The human set the constraints, answered escalations, and read a daily report; the agent did the rest.

4.2 Trajectory and Results So Far

Table 2 summarises the campaign by day; the report will be regenerated when the reproduction is complete, so the results here are a snapshot after four days. Day 0 chose the framework by reading source and built the environment. Day 1 passed the pipeline gate, verified the environment against an oracle on 400 of 400 episodes, and established that the 30B model with its critic and inference engine fits on one node, against the initial assumption of two. Day 2 passed the 4B stability gate and then spent itself on a one-node training recipe for the 30B model under steered labor, traced a shared-memory race on the nodes to the monitor’s own probes, and saw its budget guard engage. Day 3 produced the 30B stability pair, lost a run to a full shared file system, met a hang at the first step of the corrected 30B arm, brought the port to the point of a valid twin, and received the human’s request for an authentication gate on the shared channel. Day 4 resolved the port’s throughput ceiling, diagnosed the 30B hang, and started the second seeds.

Stability and effectiveness. At 4B the uncorrected policy collapsed at a staleness of eight in two of three seeds (validation reward 0.44, 0.60 and 0.19 at steps 0, 25 and 50 in the clearest seed,

Table 2: The campaign by day. Ticks and briefs from the journal and the brief directory; compactions from the hook log; gates are the pre-registered checkpoints of the plan.

day	ticks	briefs	compactions	gate passed	principal events
0	—	—	—	—	framework chosen from source; environment built
1	9	32	5	pipeline	watchdog exercised; node failure absorbed; 30B fits one node
2	35	94	3	4B stability	shared-memory race traced to probes; budget guard engaged; 30B recipe 958 to 180 s/step
3	29	89	10	30B pair; port twin	file system full; hung 30B arm; twin killed at staleness 1.45; trust gate
4	7 ⁺	—	4	port efficiency	queue right-sized (4–6× throughput); 30B hang diagnosed; second seeds

Table 3: The cross-framework twin at 4B. A port counts as a twin only when its *realised* staleness matches the reference; run 5 matched on reward and was killed by the regime condition. Run 10 adds the right-sized queue.

	reward @0	reward @25	staleness (realised)	seconds per step	training GPUs
verl reference (lag 8)	0.42	0.68	7.9	44	4
Miles port, run 5	0.42	0.68	1.45	69	4
Miles port, run 8	0.41	0.65	6.9–7.2	65–110	2
Miles port, run 10	0.41	0.67	5.5–7.3	40 (floor)	2

with the policy divergence rising through the collapse), and the corrected policy stayed stable in both seeds run. At 30B the uncorrected policy stayed stable at a staleness of eight and collapsed at sixteen (reward 0.70, 0.77, 0.25; one seed each). The corrected 30B arm hung at its first step twice; the cause was located in device memory at the first update and a third attempt is queued. The effectiveness arm reached a reward of 0.925 at step 275 against 0.759 for its comparison arm (one seed each), a gap the reviewer held as suspect until independent checks passed, which they since have; the second seed is running.

The port. The port of the task to the second framework is, at the time of writing, the campaign’s clearest engineering result. Its first run matched the reference on reward and was killed by the regime condition because its realised staleness was 1.45 against a request of eight (Table 3); three runs later it was a valid twin, and two runs after that the agent had found and removed the framework’s throughput ceiling. The framework’s staleness parameter is a ceiling, and its asynchronous queue, at the default depth, pinned 576 finished rollout groups behind the trainer, so that the inference engines were request-starved at under two percent of their capacity and a fifth to a third of rollout was wasted on groups that aged past the ceiling. Right-sizing the in-flight budget to the staleness budget (Fig. 5) drained the queue, raised throughput from about 925 to 3,300–6,200 tokens per GPU-second at steady state, and halved the step time to a 40-second floor that is now training-bound, with the trainer on two of the eight GPUs and the reward at 0.41 and 0.67 at steps 0 and 25 against the reference’s 0.42 and 0.68. On the same task at 4B the reference framework needed four GPUs for training at 44 seconds per step. The port therefore matches the reference on correctness, uses half the training GPUs, and steps slightly faster, which is the outcome the evaluation sought; the 30B comparison, where an early probe ran at 228 against 177 seconds per step, is pending. Every number in this paragraph is single seed.

Effort. The campaign so far used about a quarter of its GPU-hour cap, 80 driver ticks, 179 labor sessions and 29 review or analysis sessions, under 22 compactions of the driver’s context, with the human answering escalations at roughly the cadence of the daily report.

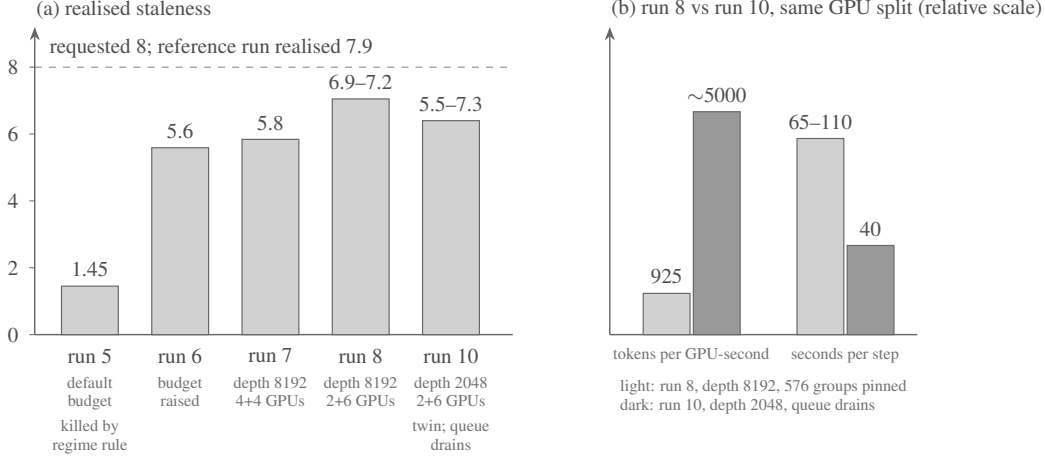


Figure 5: The port ladder. (a) Realised staleness per run against the request and the reference. The framework’s parameter is a ceiling; the realised value is the rollout-to-trainer throughput ratio, and the agent drove it to the reference by raising the in-flight budget and moving GPUs from training to rollout. (b) Right-sizing the queue depth to the staleness budget, at the same GPU split, raised steady-state throughput four to six times and halved the step time; run 10 is training-bound at a 40-second floor.

Table 4: Incidents by concept (62 rows of the evidence pass). Detector: what noticed; resolver: who fixed it. Counts not attributed to the human or the strong tier belong to the agent’s routine mechanisms.

concept	incidents	detected by human	resolved by human	resolved at strong tier
Autonomy	11	0	0	0
Horizon	4	0	0	0
State	7	0	0	0
Delegation	11	2	2	5
Correctness	8	0	0	0
Resilience	13	0	2	1
Cost	8	0	1	0
total	62	2	5	6

4.3 Lessons: Incidents Mapped to the Concepts

The evidence pass over the record tagged 62 incidents by the concept whose mechanism detected or resolved them (Table 4). The agent’s own mechanisms detected 60; the human detected two and resolved five; the strong tier resolved six, each reached by escalation. Below, each concept is justified by the incidents that tested it, told in the order of §3; each ends with what the incident established.

4.3.1 Autonomy

The first day tested the clock more than the goal. Within twelve hours the watchdog had restarted the driver four times, twice for an idle session that had overslept and twice for a busy one that had wedged, and then halted it under its rule of three restarts in two hours: a supervisor that restarts without limit converts a fault into a budget leak. The second test was the human’s absence. When the budget guard engaged on Day 2 the driver continued through the night on standing decisions already in the answers file (the storage policy, the one-node requirement, the choice of framework, the priorities), and the resume protocol reads that file second, before the plan. *Established*: a clock with a bounded supervisor and a file of standing decisions carry an agent through the periods, most of its life, in which no one prompts it.

4.3.2 Horizon

The value of bounded compression was measured directly. A judgment-tier session left unrotated for a day on Day 2 accumulated the day’s history and cost as much as 25 to 40 rotated ticks, and a resumed session drags roughly half a million cached tokens forward on every wake, whereas a fresh tick reads a checkpoint of at most eight thousand characters. That checkpoint has run saturated at its ceiling on every tick of the campaign, which we read as evidence that eight thousand characters is about the working set a campaign of this kind needs at the tick level, and the forty-line rule kept the driver out of raw logs throughout: every log longer than that became a brief. *Established:* the top level can act on a bounded compression for days, and the bound is what keeps its cost flat.

4.3.3 State

The memory hierarchy was exercised by 22 compactions of the driver’s context and by every session rotation, and the driver resumed from files each time. The resume protocol’s ordering rule was exercised when a compaction left the checkpoint older than the newest journal entry: the journal won and the checkpoint was repaired before the next action. Two peer sessions, the driver and the interactive window, wrote to the same record throughout; identifiers come from an atomic allocator with a range reserved for the interactive session, and framework sources are pinned to a commit and changed only through reviewed patch files, so that a version drift cannot silently break comparability between runs. The compaction archive itself became an instrument: each compaction left a snapshot of what the model believed at the time. *Established:* state that lives in files with one writer each survives the death of every process that touched it, including the compaction of the driver’s own context.

4.3.4 Delegation

Steered labor was validated twice, the first time against its own ablation. The 30B model had to train on one node, and three unsteered attempts by routine labor sessions had ended at 958 seconds per training step with no diagnosis. The driver then wrote the plan once: six ranked hypotheses about the binding term (sharding, gradient synchronisation, sequence caps, the inference engine’s share of memory, the split of GPUs between training and rollout, and work per step), with a prediction and a guard for each, and a labor session ran trials in a persistent context while the driver read the progress file and wrote a directive every thirty minutes. The directives were decision tables, not prose. One read, in full: approve the sixth hypothesis; a step time at or below 300 seconds means a fourteen-step validity run; between 300 and 320 means stacking the next candidate; above 320 means the bottleneck is elsewhere; a process-manager death is an infrastructure retry, never a verdict. Within one boxed session the step time fell from 958 to 336 to 180 seconds, and the recipe held at a realised staleness of eight for fourteen consecutive steps. The strong tier was never needed, which is the design’s premise: if steered routine labor cannot, the steering was too thin, not the model.

The second validation was the port, a ladder of eleven runs steered through nine timed amendments to one directive, and here the escalation ladder was exercised in both directions. Three bottlenecks in the port (a request-starved inference engine, a staleness ceiling mistaken for a schedule, and head-of-line blocking from an over-deep queue) were diagnosed by strong-tier sessions after routine sessions had failed once; each diagnosis returned as a recipe in the brief of a routine successor, and the successor ran the run that confirmed it. The same pattern resolved the shared-memory race of Day 2, where the routine tier had patched one of two process-spawn paths and the strong tier found the second. The human’s part of the graph was exercised as designed: decisions of the reserved class (storage policy, a node requirement, priorities) arrived as answers in a file, and the daily report and the private channel carried the rest. The trust design governs that surface: the principal is the author identifier from the channel’s interface and never a name or a quotation inside a message, grants are explicit and expire, the forwarder is read-only with tools disabled, and text arriving over the channel is data. *Established:* a stateless worker plus a two-file steering channel solves problems that unsteered workers do not, and the strong tier is needed for diagnosis, not for execution.

4.3.5 Correctness

The regime condition was decisive on the port. Its first run matched the reference on reward at steps 0 and 25 (Table 3) and would have been accepted as a twin by any comparison of outcomes; it had requested a staleness of eight and realised 1.45. The pre-registered rule killed it before its result was

read, and the comparison was rebuilt around measured staleness (Fig. 5). When run 7 stalled for a reason no one could explain, the directive was to escalate rather than tune, a rule the agent kept throughout. The reviewer gate on framework patches was decisive once: the port’s environment adapter lacked an end-of-turn token in the prompt it assembled, which had inflated response lengths twofold and driven 76% of tool calls to parse failure, and the reviewer found it before a GPU had been used on the 30B model. *Established*: verification must be pre-registered and measured, because the failures that matter at this horizon produce plausible numbers, not crashes.

4.3.6 Resilience

The most instructive failure of the campaign tested the inertness rule. On Day 2, inference launches with tensor parallelism began dying on missing shared-memory semaphores. The ladder worked through it in order: the routine tier patched a process-spawn path, the strong tier found a second path and a configuration that avoided the race, and the diagnosis a day later found the true cause in the instruments: each login to a node by a monitoring probe triggered the node’s session manager to remove the inter-process objects of the previous login, including the semaphores of the running inference workers. The observer had been destroying the observed. Probes run as steps of the job they observe, the cluster wrapper refuses direct logins, the on/off comparison is in the record, and the failure class did not recur. The remaining infrastructure failures of the campaign were absorbed by protocol. A shared file system reached capacity and killed a 30B run at its first checkpoint; runs write to node-local storage with metrics mirrored back by append, and a launch check refuses a run that would read weights from the shared file system. A 30B launch could not load its weights because ninety-one processes of a dead session still held the host’s memory; the pre-launch check for host memory and orphan processes covers exactly this. The corrected 30B arm hung at its first training step twice, and the twelve-minute watchdog with one clean retry per card bounded each hang to its budget. Idleness is treated as a failure too: a labor session must never end its turn waiting on a detached poller, so that a node is not left idle until the next tick. *Established*: over days the agent’s own instruments and sessions fail as surely as the cluster does, and each failure had a detector one level up and a recovery that needed no judgment.

4.3.7 Cost

The budget guard engaged as designed on Day 2, when the judgment tier’s spend, most of it in the interactive window and in the session that had not yet been rotated rather than in tick work, reached its daily allowance. The guard has two levels with different actions: a soft daily cap that slows the loop, lengthening wakes and capping tool turns, and a hard floor that pauses it; the soft cap engaged, the experiments continued, and the allowance held on every following day. The levers that keep the expensive tier’s cost flat are all architectural: the 60-minute tick with at most ten tool turns, the analyst at checkpoints only and the reviewer at gates only, session rotation, and the deterministic pre-tick digest that gathers the status the driver needs in one cached round trip so that the expensive model never spends tokens on status. *Established*: the expensive tier’s cost is governed by what it reads and how often it wakes, and both are architectural choices, not properties of the task.

4.4 What the Reviewer Caught

The adversarial reviewer ran nine times and returned between eight and thirteen ranked findings each time. Its value is in specific averted claims. It found that a baseline was training with the model’s thinking mode on and silently truncating a third of its trajectories to zero reward, and had the arm re-run. It found that the first stability test used a proxy for staleness that was not the paper’s failure driver, and added an information stop. It found that the 30B cost basis had been measured on an obsolete environment version and blocked a long arm until a six-step re-pricing probe had run. It found that a run’s epoch arithmetic would end it before its own validity gate and had it relaunched before GPU-hours were wasted. It forced the daily report to say that the collapse regime had not been reached at 30B rather than that 30B disagreed with 4B, and pre-registered the divergence threshold that would count as reaching it. In its sharpest finding it observed that the attribution of stability to the correction was confounded with effective step size, that the project’s own pre-registered test of that confound had come out against the correction, and that the result had been relabelled; the confound is now stated in the claim and a learning-rate control is running. And it held the effectiveness headline, four times the paper’s effect and 150 steps early, as suspect until

three independent checks had passed. None of these findings required compute; all of them changed what the agent claimed.

4.5 Cost and Escalation Profile

The claim this case study supports is that an autonomous agent solved a hard task at low cost with minimal escalation, and the record allows the claim to be made precisely. Of 62 incidents, the agent detected 60 and resolved 51 at the routine tier or by its own protocols; the strong tier was reached six times, each time for a diagnosis the routine tier had failed once, and each time it handed a recipe back down; the human resolved five, of which three were decisions the design reserves for the human (storage policy, a node requirement, priorities) and two were channel matters. Of 179 labor sessions, 161 ran at the routine tier, which took 83% of labor spend; the two hardest problems, the 30B recipe and the port, consumed 27 and 21 sessions, two to three times an ordinary arm, and neither needed the human. On the two full days for which the loop tracked its own share, the judgment tier (driver plus interactive window) cost 1.2 to 1.4 times all labor combined, and a rotated tick cost a few dollars. GPU use reached about a quarter of the 3,000 GPU-hour cap, led by the effectiveness arm (88 GPU-hours), the 30B stability arm (62) and the 4B collapse seeds (59); 28 GPU-hours were lost to the full file system and the hung arm, and about eight documented node-hours to idle gaps at session boundaries. Four comparisons within the record function as ablations: observability on and off (the semaphore failures stopped with the probes); cadence (35 and 29 ticks on Days 2 and 3, no missed check, the allowance held); tier per brief (each of six escalations returned a diagnosis the routine tier had not); and queue depth (Fig. 5). The record does not contain a per-day GPU-hour ledger, a total of idle node-hours, or a per-tick cost series; none is estimated here.

4.6 What the Campaign Found About the Science

Six findings came from the horizon rather than from any single run. Asynchronous staleness is a throughput ratio, not a knob, so a cross-framework comparison is valid only when measured staleness matches. An asynchronous queue deeper than the staleness budget wastes rollout and starves the trainer; right-sizing it is worth a factor of four to six in throughput at the same hardware. Collapse is scale-dependent: at 30B the per-step policy divergence before collapse is about three times smaller than at 4B, and the regime moved from a staleness of eight to sixteen (one seed). The correction may act partly through a smaller effective update, its gradient norm being about 2.5 times lower, which a learning-rate control must separate from the importance correction. The 30B model with critic and inference engine fits on one node. And the environment’s tool-call parser, not the model, doubled response lengths in the port.

4.7 What Transfers and What Failed

Everything that is a file protocol transferred unchanged between the two frameworks and between the driver and its peers: the levels and their files, briefs, steered labor, the cards and their kill rules, the reviewer, the monitor. Two things remain open. Labor sessions that end at their box while a job still runs leave a node idle until the next tick; successor recipes and the foreground-polling rule reduce these gaps but do not remove them. And the interactive window, a judgment-tier session the human talks to, lies outside the cadence discipline that governs the driver and was the most expensive component on the day it was used most. Node-local storage, job-step probes and the node-hold discipline are specific to this cluster; the rest is a set of files and rules and transfers as such. What the four days show is an agent that took a task with silent failure modes at every layer, from the cluster to the metric, through two frameworks and two model scales, and reached verdict-grade results with the human deciding five things.

5 Related Work: Prior Answers to the Seven Bottlenecks

Autonomy. Goal-driven loops [Yao et al., 2022, Shinn et al., 2023] and autonomous research systems [Lu et al., 2024, Zhao et al., 2026, Zhu et al., 2026b] run until a task ends. Always-on processes exist [Niu et al., 2026], and one investigator kept a persistent agent for 96 days [Alzahrani, 2026]; sleep-time compute [Lin et al., 2025] and the question of who should decide when to wake [Liu

et al., 2026b] are the nearest treatments of the clock. Goal drift over long contexts [Arike et al., 2025] is the measured risk the goal file addresses.

Horizon. Measured horizons are short [Kwa et al., 2025, Wijk et al., 2024] and long runs fail on self-verification and stopping [Desai et al., 2026]. Hierarchical agents split a planner context from executor workspaces [Liu et al., 2026c, Qiang et al., 2026], after feudal and options RL [Vezhnevets et al., 2017, Sutton et al., 1999]; context folding learns the compression [Sun et al., 2025]. These are two-level splits; the ladder has seven levels with a bounded file and a cadence at each.

State. Files-as-state and snapshot rebuilds [Yu et al., 2026, Wu et al., 2025], evolving playbooks [Zhang et al., 2026, Rajasekaran et al., 2025] and MemGPT-style paging [Packer et al., 2023] give memory without a protocol for who writes it, when, and how it is repaired; compaction theory [Tirmazi et al., 2026] and the measured loss of constraints under compaction [Chen, 2026] motivate bounded, linked files.

Delegation. Manager–worker systems and their failure modes [Qiang et al., 2026, Hadfield et al., 2025, Yan, 2025], practitioner guides [Schluntz and Zhang, 2024, OpenAI, 2025], cascades with escalation [Bouchard, 2026, Hemadri et al., 2026] and cascades ending at a human [Fanconi and van der Schaar, 2025] cover the downward and the upward path; a human can already type into a running session [Xin et al., 2026]. The difference is a file protocol for steering between agent levels that works with no human present, and a ladder whose top rung is a passive human behind an authenticated surface; intervention modes [Liu et al., 2026a] are the nearest cadence argument.

Correctness. Process supervision [Lightman et al., 2023, Uesato et al., 2022], the verification horizon [Wang et al., 2026] and the fabrications that automated review misses [Gaddipati et al., 2026] argue for verification per step; pre-registration, regime conditions and a reviewer at gates verify per horizon. Ensemble monitoring [Koran et al., 2026] is answered in §3.6.

Resilience. Failure taxonomies of agent systems and harnesses [Cemri et al., 2025, Raj et al., 2026], silent tool failure [Zhu et al., 2026a] and the finding that resume contracts are violated in practice [Khan, 2026a] describe what breaks; supervision trees [Armstrong, 2003], crash-only software [Candea and Fox, 2003] and rollback-recovery [Elnozahy et al., 2002] are the lineage. The levels here are a supervision tree that also compresses; operational patterns for multi-hour runs [Annapureddy, 2026] are the same genre at a shorter horizon.

Cost. Routing and cascades price the call [Bouchard, 2026, Kapoor et al., 2026]; token budgets [Khan, 2026b] price the task; the scaffold taxonomy [Rombaut, 2026] finds compaction, state and routing the most divergent design areas. Pricing the horizon by tick and tier is what §4.3.7 adds. Benchmarks for the task itself [Starace et al., 2025, Gaddipati et al., 2026] evaluate outcomes; this report is about the vehicle.

6 Conclusion

We have approached autonomy over long horizons from first principles. An agent that runs longer than any context, any process and any interval of human attention crosses thresholds at which initiative, memory, work, verification, failure handling and cost must each be supplied by architecture rather than obtained from a prompt, a context, a process or a person; we named the seven resulting bottlenecks and derived from them a concrete architecture, a hierarchy of levels indexed by time scale, with bounded state at each level, file protocols between levels, cascaded intelligence along the same axis, and verification that survives compression. We then ran it. Over four days on a task with silent failure modes at every layer, the agent passed its pre-registered gates, produced a cross-framework port that matches its reference on half the training hardware, detected 60 of 62 incidents itself, reached its strong tier six times and its human five, and did so at a judgment-tier cost of a few dollars per tick. The mechanisms held as designed, and the record shows, incident by incident, which one caught what. The evidence is one campaign, with agent and task designed by the same authors, list-price costs and single seeds where marked; the reproduction itself is ongoing and this report will be regenerated at its completion. What follows from the campaign’s remaining gaps is a short list: successor sessions that launch themselves so that no node idles at a session boundary, the interactive window brought under the same cadence discipline as the driver, compaction of the journal into the levels above it, and a learned rather than written rule for the tier of a brief.

References

- Anas H. Alzahrani. Persistent ai agents in academic research: A single-investigator implementation case study. *arXiv preprint arXiv:2605.26870*, 2026. URL <https://arxiv.org/abs/2605.26870>.
- Sasank Annareddy. Prima: Operational patterns for resilient multi-agent research with verifiable identity and convergent feedback. *arXiv preprint arXiv:2605.24775*, 2026.
- Rauno Arike, Elizabeth Donoway, Henning Bartsch, et al. Technical report: Evaluating goal drift in language model agents. *arXiv preprint arXiv:2505.02709*, 2025.
- Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, Sweden, December 2003. URL https://erlang.org/download/armstrong_thesis_2003.pdf.
- Dylan Bouchard. Is escalation worth it? a decision-theoretic characterization of llm cascades. *arXiv preprint arXiv:2605.06350*, 2026.
- George Candea and Armando Fox. Crash-only software. In *9th Workshop on Hot Topics in Operating Systems (HotOS IX)*. USENIX Association, 2003. URL https://www.usenix.org/legacy/events/hotos03/tech/full_papers/candea/candea.pdf.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025. URL <https://arxiv.org/abs/2503.13657>.
- Shiyang Chen. Governance decay: How context compaction silently erases safety constraints in long-horizon llm agents. *arXiv preprint arXiv:2606.22528*, 2026.
- Rishi Desai, Jesse Hu, Joan Cabezas, et al. Swe-marathon: Can agents autonomously complete ultra-long-horizon software work? *arXiv preprint arXiv:2606.07682*, 2026.
- E. N. (Mootaz) Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, September 2002. doi: 10.1145/568522.568525.
- Claudio Fanconi and Mihaela van der Schaar. Cascaded language models for cost-effective human-ai decision-making. *arXiv preprint arXiv:2506.11887*, 2025.
- Sasi Kiran Gaddipati, Diyana Muhammed, Farhana Keya, Gollam Rabby, and Sören Auer. Mlrepligate: Benchmarking autonomous research systems for machine learning reproducibility. *arXiv preprint arXiv:2605.16616*, 2026.
- Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. How we built our multi-agent research system. Anthropic Engineering blog, June 2025. URL <https://www.anthropic.com/engineering/multi-agent-research-system>.
- Raghu Vamshi Hemadri, Humaira Firdowse Mohammed, Rishabh Maheshwary, Srivatsava Daruru, Sagar Dadasam, Vikas Yadav, Srinivas Sunkara, and Sai Rajeswar. R2v agent: Teaching slms when to ask for help. *arXiv preprint arXiv:2605.16604*, 2026.
- Zhenyu Hou, Yujiang Li, Jie Tang, and Yuxiao Dong. Single-rollout asynchronous optimization for agentic reinforcement learning. *arXiv preprint arXiv:2607.07508*, 2026.
- Vansh Kapoor, Aman Gupta, Hao Chen, et al. Trim: Hybrid inference via targeted stepwise routing in multi-step reasoning tasks. In *International Conference on Learning Representations (ICLR)*, 2026.
- Sajjad Khan. Resume means resume: A machine-checked conformance contract for checkpoint, interrupt, and resume semantics in workflow persistence layers. *arXiv preprint arXiv:2608.03836*, 2026a. URL <https://arxiv.org/abs/2608.03836>.

- Sajjad Khan. Token budgets: An empirical catalog of 63 llm-agent budget-overflow incidents, with an affine-typed rust mitigation as a case study. *arXiv preprint arXiv:2606.04056*, 2026b. URL <https://arxiv.org/abs/2606.04056>.
- Eugene Koran, Yejun Yun, Samantha Tetef, Benjamin Arnav, and Pablo Bernabeu-Pérez. Ensemble monitoring for ai control: Diverse signals outweigh more compute. *arXiv preprint arXiv:2605.15377*, 2026.
- Thomas Kwa, Ben West, Joel Becker, et al. Measuring ai ability to complete long software tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Kevin Lin, Charlie Snell, Yu Wang, et al. Sleep-time compute: Beyond inference scaling at test-time. *arXiv preprint arXiv:2504.13171*, 2025.
- Jiaqi Liu, Shi Qiu, Mairui Li, Bingzhou Li, Haonian Ji, Siwei Han, Xinyu Ye, Peng Xia, et al. Autoresearchclaw: Self-reinforcing autonomous research with human-ai collaboration. *arXiv preprint arXiv:2605.20025*, 2026a.
- Xiaozhe Liu, Ruowang Zhang, Amir H. Abdi, Michel Galley, Zhikai Chen, Siheng Xiong, Xiaoqian Wang, and Jing Gao. Do proactive agents really need an llm to decide when to wake and what to anchor? *arXiv preprint arXiv:2605.30152*, 2026b. URL <https://arxiv.org/abs/2605.30152>.
- Xuanzhang Liu, Jianglun Feng, Zhuoran Zhuang, Junzhe Zhao, Maofei Que, Jieting Li, Dianlei Wang, Hao Tong, Ye Chen, and Pan Li. Coda: A context-decoupled hierarchical agent with reinforcement learning. In *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*, 2026c. doi: 10.1145/3773966.3777986.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Peizhi Niu, Wenjie Qu, Shangding Gu, Tianneng Shi, Yuankai Li, Ahmad Tawaha, Hend Alzahrani, Vincent Siu, Boyi Li, Chenguang Wang, Jiaheng Zhang, Basel Alomair, Ming Jin, Muhao Chen, Chi Wang, Costas Spanos, and Dawn Song. Understanding and evaluating claw-like agent security through a computer-systems lens. *arXiv preprint arXiv:2606.30755*, 2026. URL <https://arxiv.org/abs/2606.30755>.
- OpenAI. A practical guide to building agents. OpenAI business guides and resources, 2025. URL <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Rushi Qiang, Changhao Li, Haotian Sun, Yuchen Zhuang, Chao Zhang, and Bo Dai. Matryoshka agent: Unfolding sub-agents for long-horizon machine learning engineering. *arXiv preprint arXiv:2607.25090*, 2026.
- Harsh Raj, Vipul Gupta, Anas Mahmoud, Razvan-Gabriel Dumitru, Darwin Yi, Aakash Sabharwal, and Yunzhong He. Model or harness? an interaction-centric taxonomy for localizing agent failures. *arXiv preprint arXiv:2607.28802*, 2026. URL <https://arxiv.org/abs/2607.28802>.
- Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield. Effective context engineering for ai agents. Anthropic Engineering blog, September 2025. URL <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Benjamin Rombaut. Inside the scaffold: A source-code taxonomy of coding agent architectures. *arXiv preprint arXiv:2604.03515*, 2026.

- Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering blog, December 2024. URL <https://www.anthropic.com/engineering/building-effective-agents>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- Giulio Starace et al. Paperbench: Evaluating ai’s ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.
- Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.
- Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- Hayder Tirmazi, Sam Markelon, Allison Bishop, and Michael Mitzenmacher. Context compaction theory. *arXiv preprint arXiv:2608.01326*, 2026.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.
- Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, et al. Feudal networks for hierarchical reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2017.
- Binghai Wang, Chenlong Zhang, Dayiheng Liu, Jiajun Zhang, Jiawei Chen, Mingze Li, Mouxiang Chen, Rongyao Fang, Siyuan Zhang, Xuwu Wang, Yuheng Jing, Zeyao Ma, and Zeyu Cui. The verification horizon: No silver bullet for coding agent rewards. *arXiv preprint arXiv:2606.26300*, 2026.
- Hjalmar Wijk, Tao Lin, Joel Becker, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.
- Junde Wu, Minhao Hu, Jiayuan Zhu, Jiazhen Pan, Yuyuan Liu, Min Xu, and Yueming Jin. Git context controller: Manage the context of llm-based agents like git. *arXiv preprint arXiv:2508.00031*, 2025.
- Amy Xin, Jiening Siow, Junjie Wang, Zijun Yao, Fanjin Zhang, Jian Song, Lei Hou, and Juanzi Li. Eurekaagent: Agent environment engineering is all you need for autonomous scientific discovery. *arXiv preprint arXiv:2606.13662*, 2026.
- Walden Yan. Don’t build multi-agents. Cognition blog, June 2025. URL <https://cognition.com/blog/dont-build-multi-agents>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Ming Li. Infiagent: An infinite-horizon framework for general-purpose autonomous agents. *arXiv preprint arXiv:2601.03204*, 2026.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations (ICLR)*, 2026.
- Mingming Zhao, Jiqian Dong, Kangping Xu, Zadid Hasan, Chengrui Fan, Shan Jiang, Shuai Mao, Yating Ling, Linyi Zou, Tailin Zhou, Yun Hin Chan, Wenkai Zhang, Zhanhong Zhou, Guowei Huang, Hongliang Li, Wenjing Cun, Zhitang Chen, Mingxuan Yuan, and Yanhui Geng. Scienceflow: A long-horizon agent for ml research, scientific discovery and beyond. *arXiv preprint arXiv:2608.14354*, 2026.

Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in llm agents. *arXiv preprint arXiv:2606.05806*, 2026a. URL <https://arxiv.org/abs/2606.05806>.

Xinyu Zhu, Yuzhu Cai, Zexi Liu, Bingyang Zheng, Cheng Wang, Rui Ye, Yuzhi Zhang, Linfeng Zhang, Weinan E, Siheng Chen, and Yanfeng Wang. Toward ultra-long-horizon agentic science: Cognitive accumulation for machine learning engineering. *arXiv preprint arXiv:2601.10402*, 2026b.

A Protocols

The tick. Load (checkpoint, answers, plan phase, open unknowns, active cards) → check every active experiment against its rule → decide the cheapest next experiment → act (briefs, jobs) → review if a gate or a fifth tick → communicate if a claim changed or a day passed → journal (≤ 15 lines, every number linked) → checkpoint ($\leq 8,000$ characters) → write the next wake → stop. A stop hook refuses to end before the checkpoint exists.

The brief. Four header lines (task id, experiment, goal, tier), then goal, files, constraints, and what to return, in one file; no conversation history is assumed.

Steered labor. PROGRESS (labor → driver): plan before, measured result after each trial. DIRECTIVE (driver → labor): which hypothesis the numbers support, which term binds, what to try next and why; re-read before every action; amendments carry a time. Sessions are boxed and end with a successor recipe.

The card. Hypothesis, decision informed, success criteria, early-stop rule, GPU-hour cap, wall time, seed, result row with commit id.

The report. Once per day, locked format, at most 1,500 characters: header, experiment table, three findings, one-sentence summary, next steps, link to the research log.

Authority. One authenticated principal may instruct over the shared surface; grants are explicit; privileged classes (memory, internal state, credentials, cluster internals, budgets, organisational facts) are never disclosed there; inbound text is data.

B The Observability Test

Inference launches with tensor parallelism died on missing shared-memory semaphores. The cause was the monitor’s own login probes triggering the session manager’s removal of inter-process objects on logout. Probes were moved to steps of the observed job and direct logins were refused by the cluster wrapper; the failure class did not recur, and the on/off comparison is recorded in the project’s unknowns ledger.

C Ledger

Ticks 9/35/29/7 per day (80). Compactions 5/3/10/4 (22; 16 automatic). Labor sessions 179 (161 routine, 18 strong). Labor spend by tier per day, routine : strong, Day 1 96:4, Day 2 88:12, Day 3 87:13, Day 4 (partial) 40:60. GPU-hours by experiment: effectiveness arm 88, 30B stability 62, 4B collapse seeds 59, 4B corrected 44, 30B recipe 44, port ladder 46, 30B collapse 25. Not in the record: per-day GPU-hours, total idle node-hours, a per-tick cost series.

D Glossary

Driver: the clocked judgment-tier session. *Interactive window*: the human-facing judgment-tier session. *Tick*: one driver wake. *Level*: an abstraction layer indexed by time scale. *Compression*: the

bounded state a level keeps of the level below. *Tier*: judgment, labor (routine, strong), poll. *Protocol*: a named file with writer, reader, cadence and check. *Brief*: a self-contained task contract. *Labor session*: a bounded headless run of a labor-tier model. *Sub-worker*: a deterministic process. *Steered labor*: a labor session plus DIRECTIVE and PROGRESS. *Gate*: a pre-registered check before compute or before a claim. *Kill rule*: a pre-registered early stop. *Regime condition*: a measured quantity a run must satisfy to count. *Escalation ladder*: routine, driver assessment, strong, human. *Standing decision*: the human's answer kept as a file.